

System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

System Design & Diagrams

Architecture, workflows, data flows, ER model and deployment topology

Project	System Scanner Pro (v3.1)
Document	System Design & Diagrams
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. Purpose & Reading Guide	3
2. Design Goals & Constraints	3
3. Architectural Style Overview	3
4. Overall System Architecture	3
5. Module Architecture	4
6. System Flowchart - Enrollment to Insight	6
7. End-User Workflow (Administrator Journey)	6
8. Administrator Workflow (Deployment View)	7
9. Data Flow Diagram - Level 0 (Context)	8
10. Data Flow Diagram - Level 1	8
11. Database Design Overview	9
11.1 ER Diagram - Core Entities	9
11.2 ER Diagram - Organisation, Assets, Maintenance & Licenses	9
12. Request Lifecycle (HTTP Pipeline)	11
13. Authentication Flows	11
14. WebSocket Communication Flow	11
15. Background Scheduler Design	12
16. Security Architecture Layers	13
17. Deployment Architecture - Linux VPS (recommended)	13
18. Deployment Architecture - Vercel Serverless (alternative)	13
19. Component Interaction Map	14
20. Design Decisions & Trade-offs	14

1. Purpose & Reading Guide

This volume visualises how System Scanner Pro is put together. Every figure was drawn from the actual code paths described in volumes 04 and 05. Read section 4 first for the big picture, then use the remaining figures as references while reading code or operating the system. Cardinalities on ER figures use crow-foot-free notation: '1' and 'N' labels at the line ends.

2. Design Goals & Constraints

Goal - single control plane - One admin server owns identity, policy and data for all endpoints.

Goal - tiny footprint on clients - Agent limited to stdlib + websockets + watchdog.

Goal - real-time operations - WebSocket push instead of dashboard polling.

Goal - deploy anywhere - Same code runs under systemd/Daphne on a VPS or as a Vercel function.

Constraint - intern team timeline - Favoured batteries-included Django over microservices.

Constraint - serverless limits - 30 s execution cap and WSGI-only entry shaped the signed-cookie session fallback and moved long work to agents/scheduler.

3. Architectural Style Overview

Style element	Applied as
Layered monolith	Single Django project; presentation (templates/JS), application (views/consumers), domain (models/engines) and data (ORM) layers
Event-driven edges	Agent telemetry and commands travel over an authenticated WebSocket; internal pub/sub via monitoring/event_bus.py
Scheduled workers	APScheduler background jobs perform housekeeping independent of request traffic
Repository pattern (light)	Models encapsulate persistence; engines (health, anomaly, predictive, diff) are pure functions over querysets
Multi-tenant scoping	Company foreign keys scope clients, users and logs; views filter by requester company

Table 1 - Architectural style decisions

4. Overall System Architecture

MANAGED MACHINES (x N)

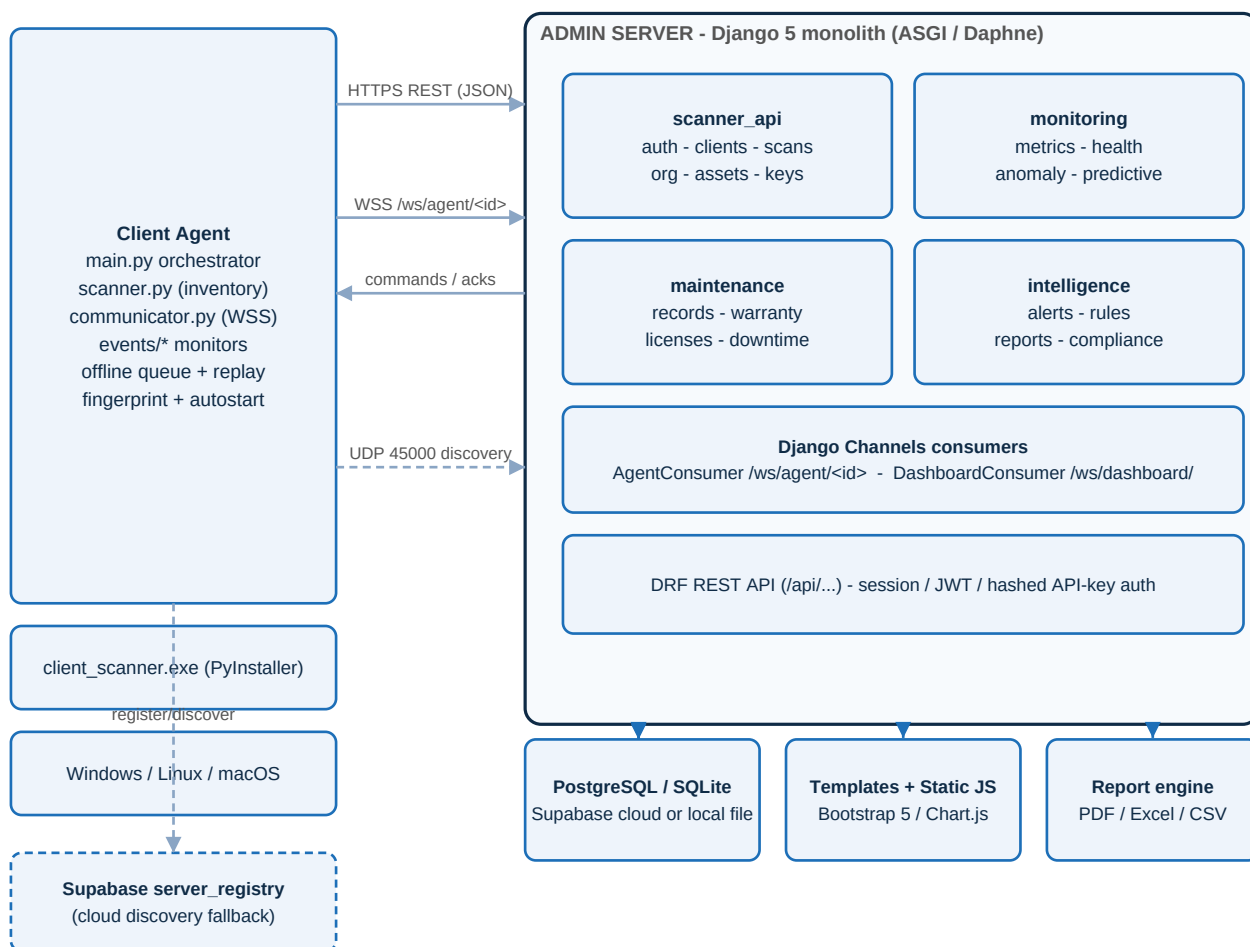


Figure 1 - Overall system architecture

5. Module Architecture

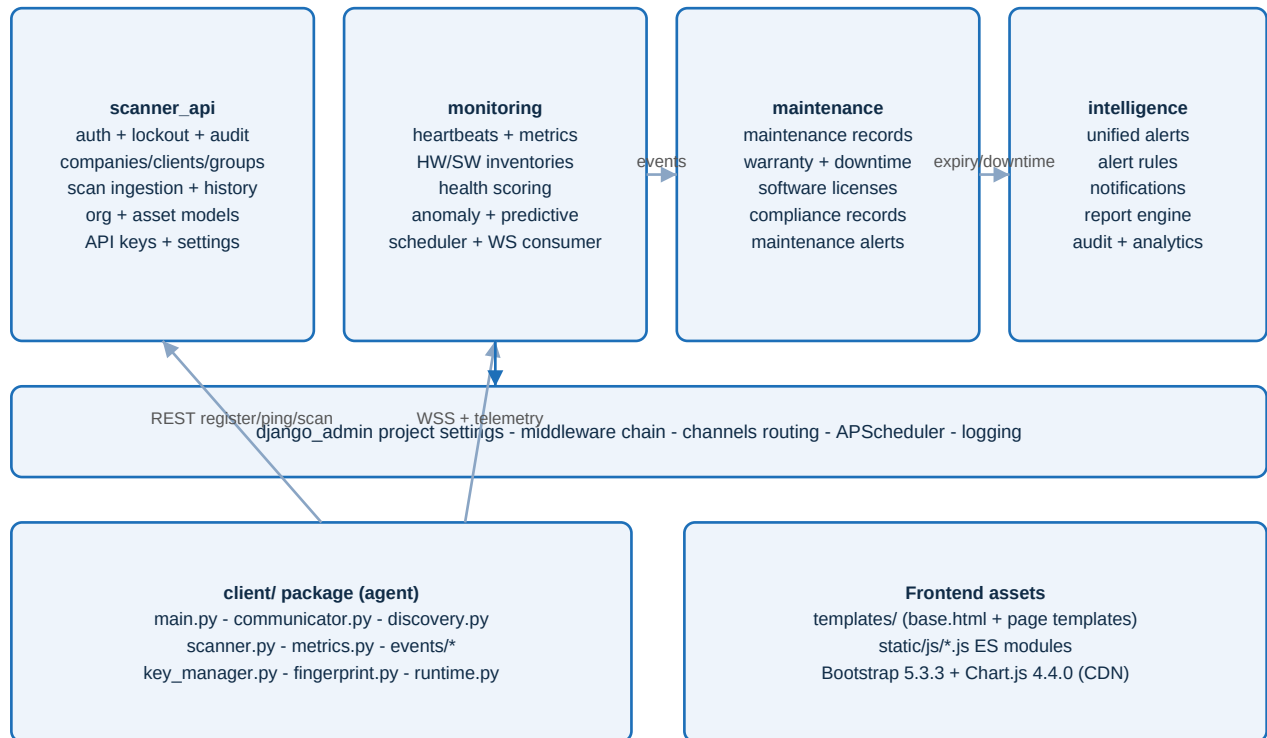


Figure 2 - Module architecture and dependencies

6. System Flowchart - Enrollment to Insight

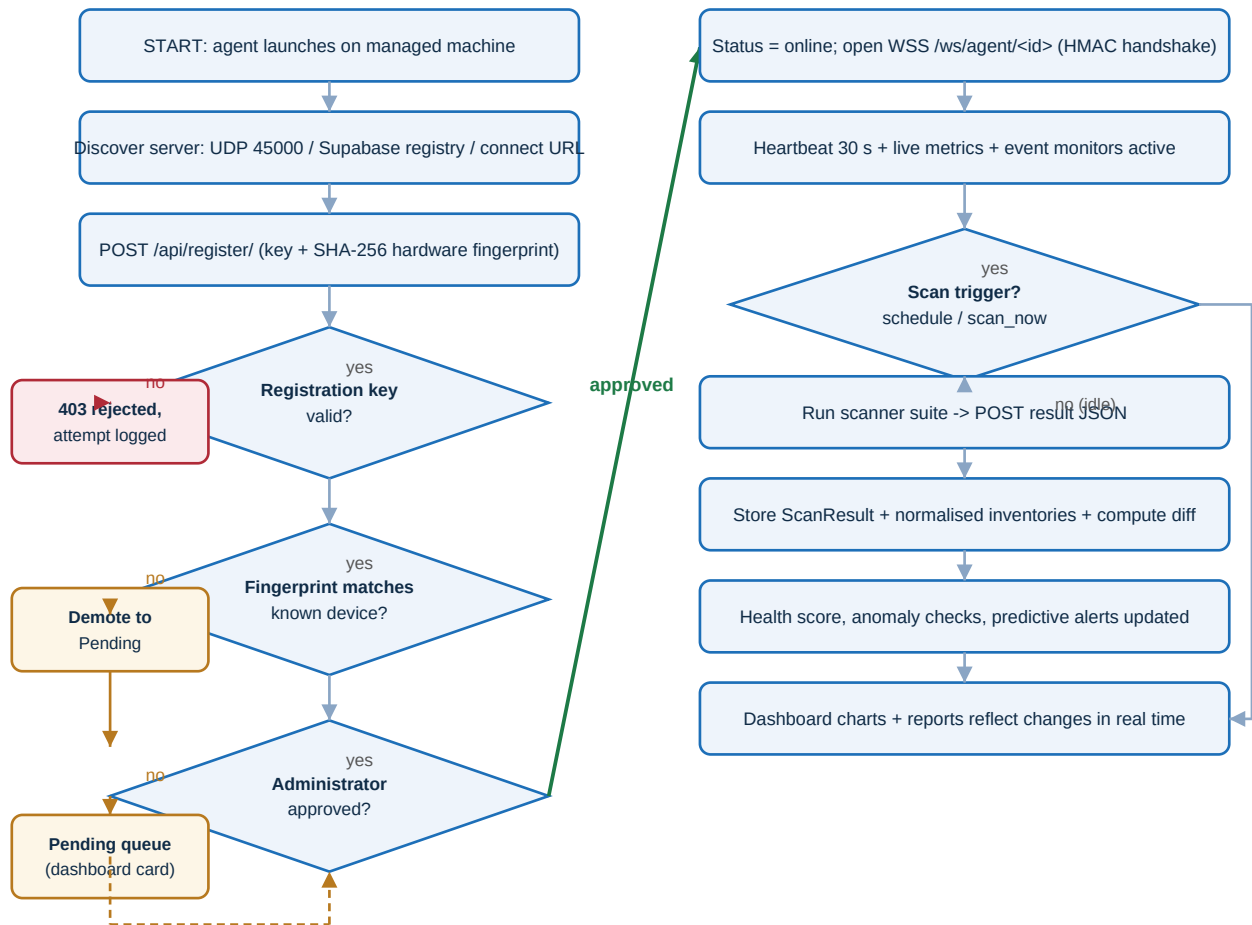


Figure 3 - End-to-end system flowchart

7. End-User Workflow (Administrator Journey)



Figure 4 - Typical administrator workflow

8. Administrator Workflow (Deployment View)

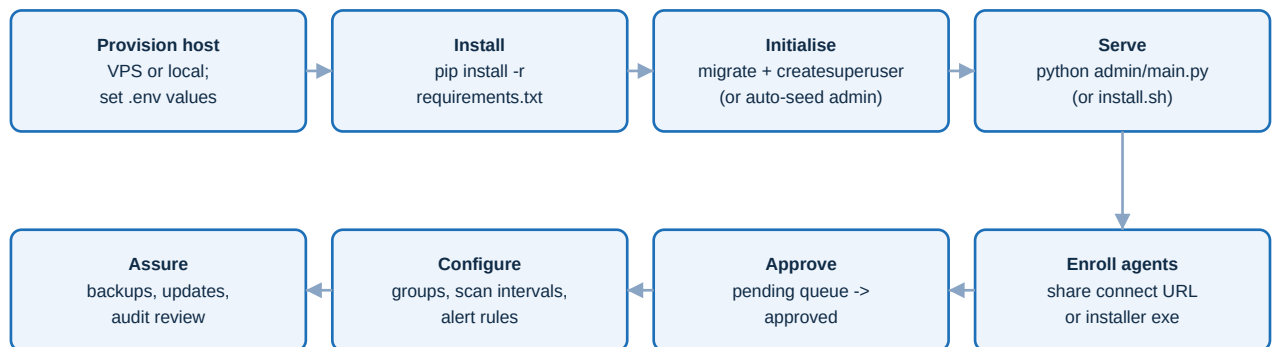


Figure 5 - Administrator deployment workflow

9. Data Flow Diagram - Level 0 (Context)

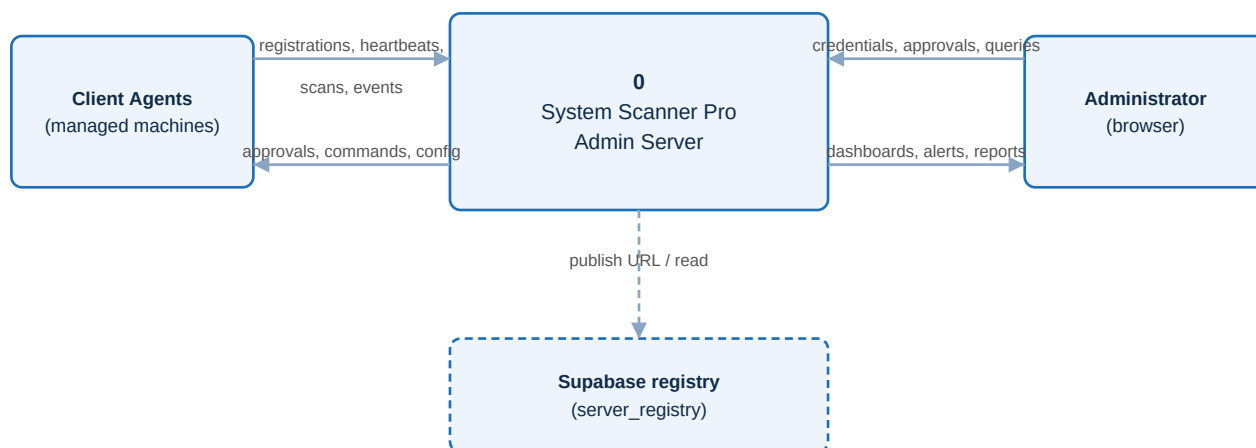


Figure 6 - Context-level data flow

10. Data Flow Diagram - Level 1

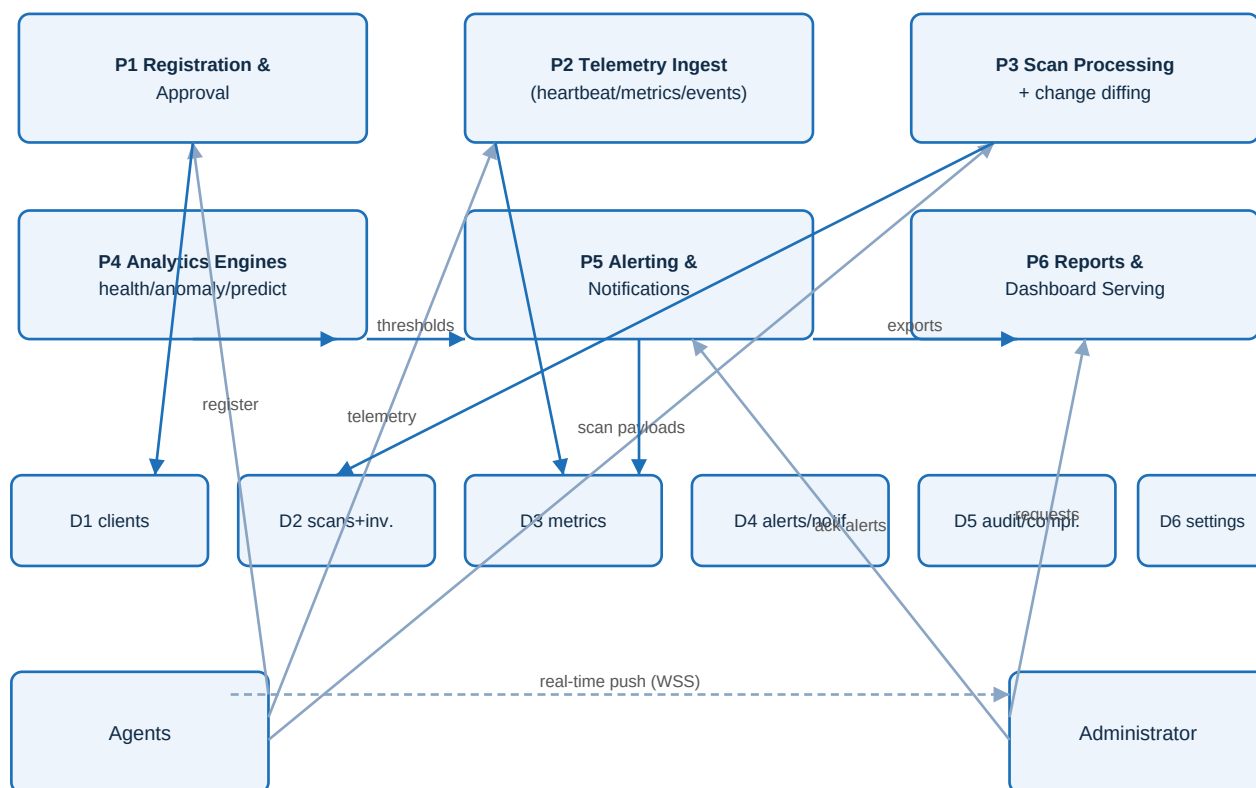


Figure 7 - Level-1 data flow with data stores

11. Database Design Overview

The schema spans roughly sixty tables across four Django apps plus Django's built-in tables (auth_user, sessions, django_admin_log, django_migrations). Two ER figures show the entities most relevant day-to-day; volume 04 lists every model per app with its purpose. Notable conventions: soft deletion via a boolean flag on Client, JSON payloads for raw scans (ScanResult.scan_data) alongside normalised inventory tables, and multiple specialised audit tables rather than a single log.

11.1 ER Diagram - Core Entities

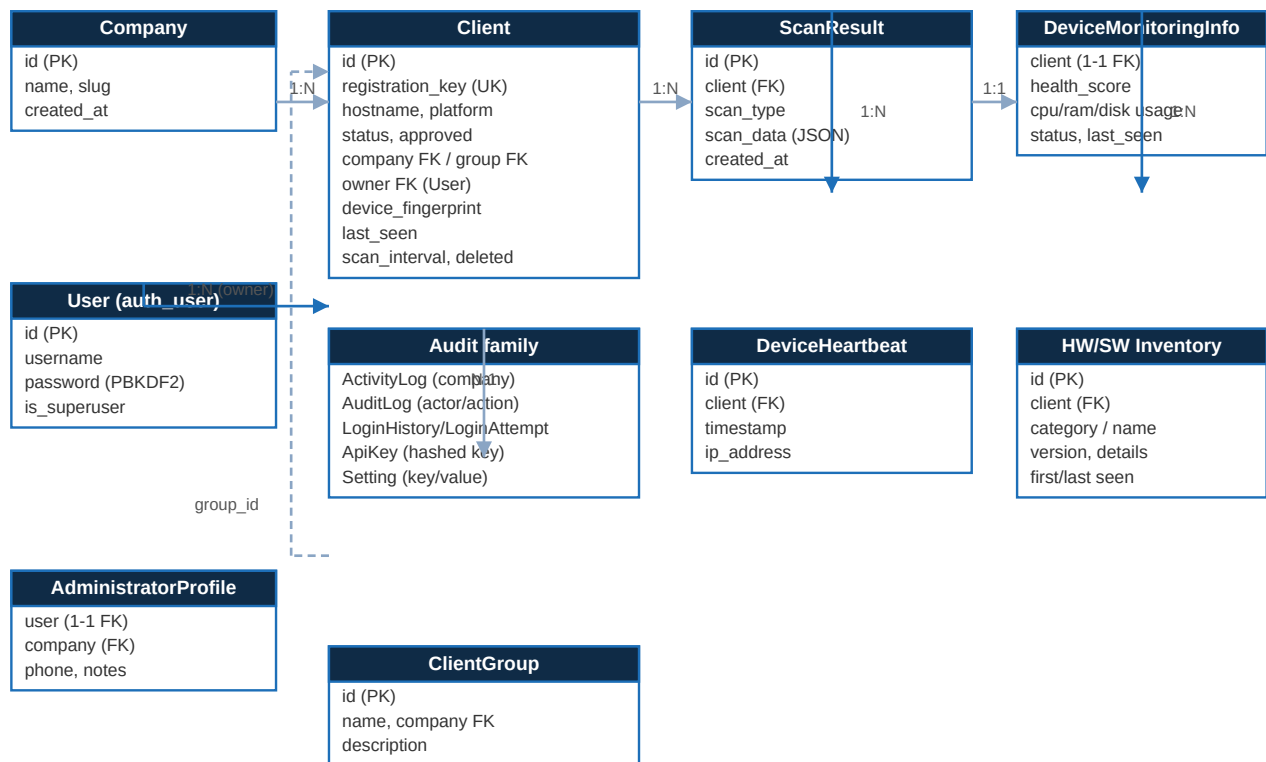


Figure 8 - Core entity relationships

11.2 ER Diagram - Organisation, Assets, Maintenance & Licenses

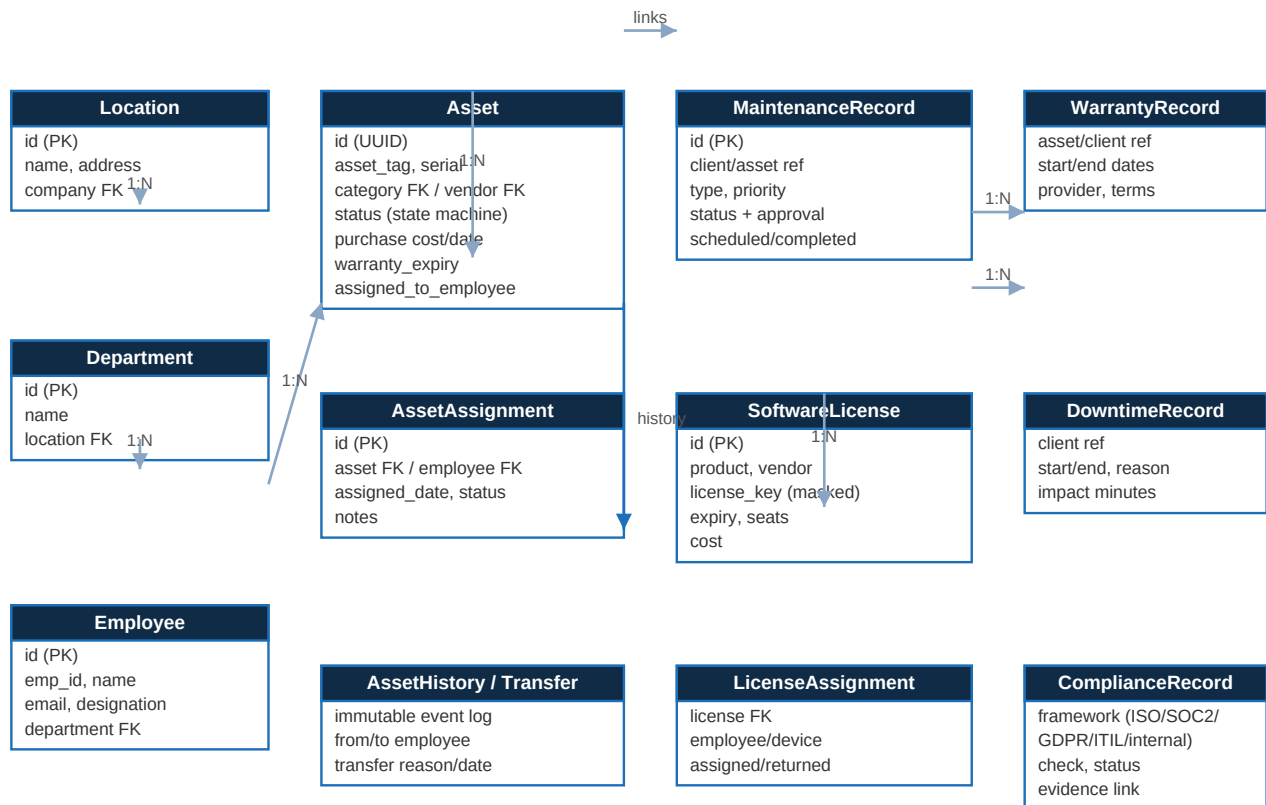


Figure 9 - Organisation, asset, maintenance and license relationships

12. Request Lifecycle (HTTP Pipeline)

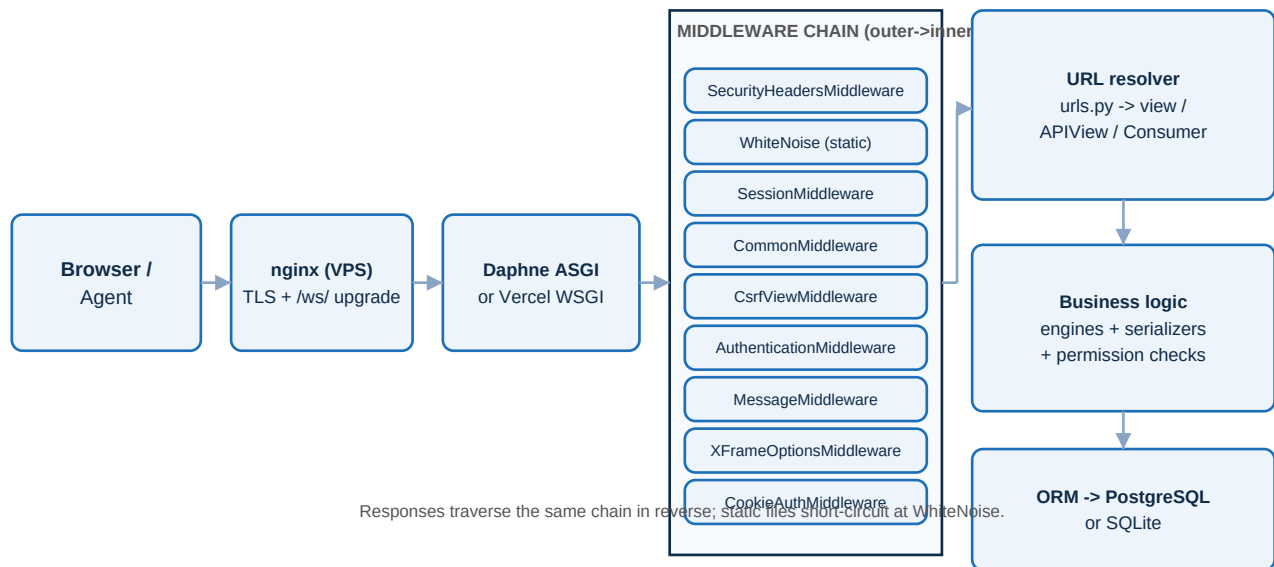


Figure 10 - HTTP request lifecycle

13. Authentication Flows

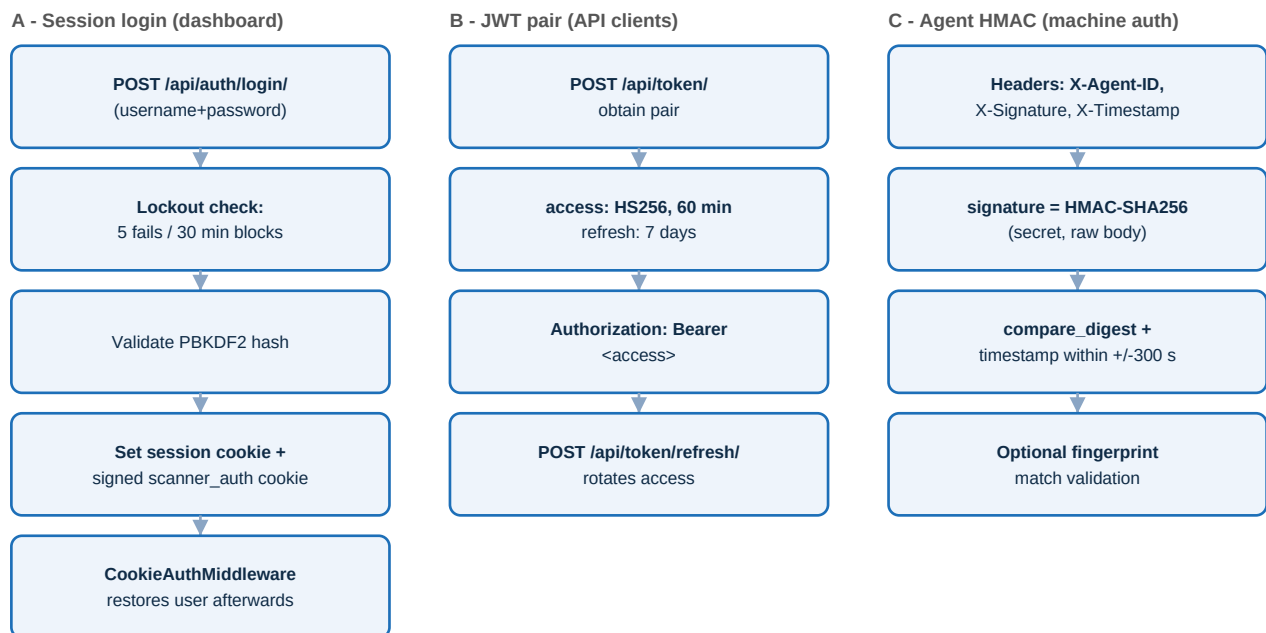


Figure 11 - Three authentication mechanisms

14. WebSocket Communication Flow

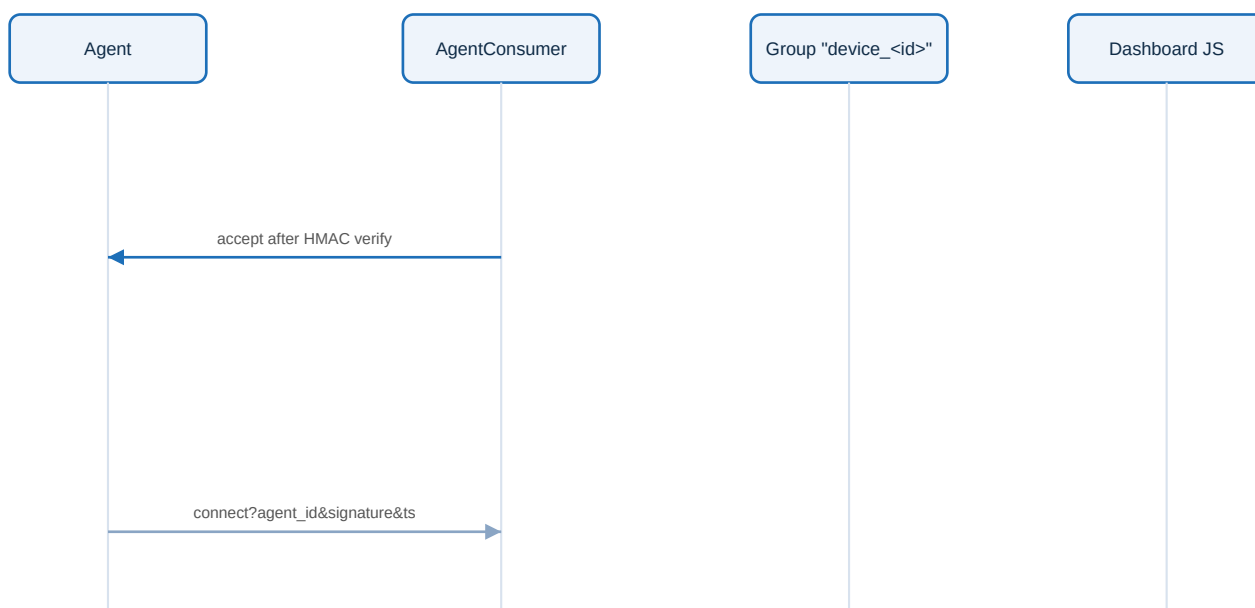


Figure 12 - WebSocket message sequence

15. Background Scheduler Design

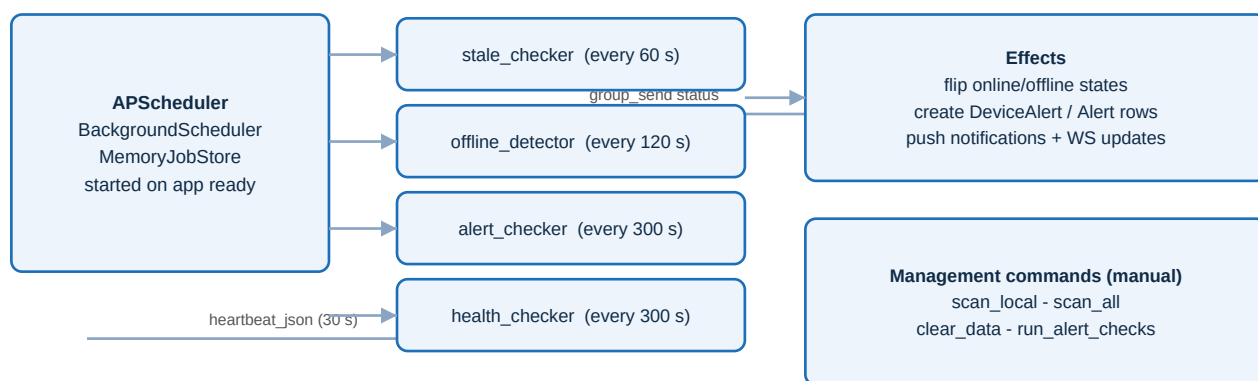
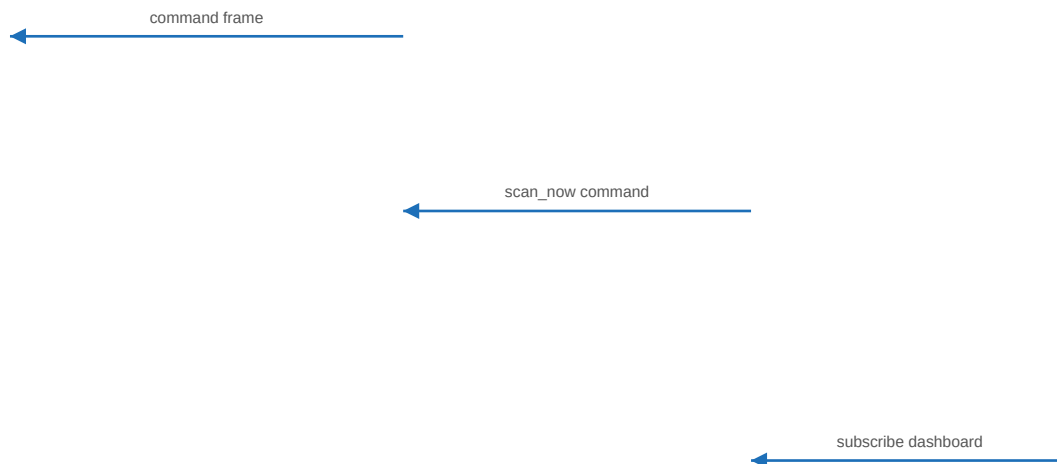


Figure 13 - Scheduler jobs and effects



16. Security Architecture Layers

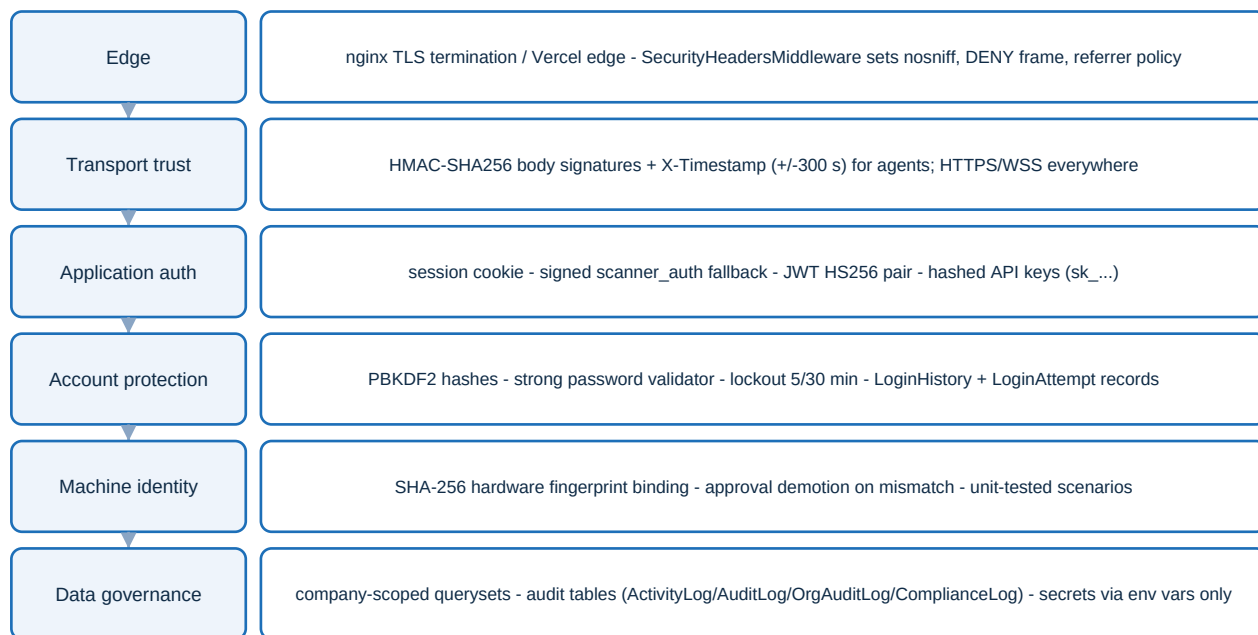


Figure 14 - Defence-in-depth layering

17. Deployment Architecture - Linux VPS (recommended)

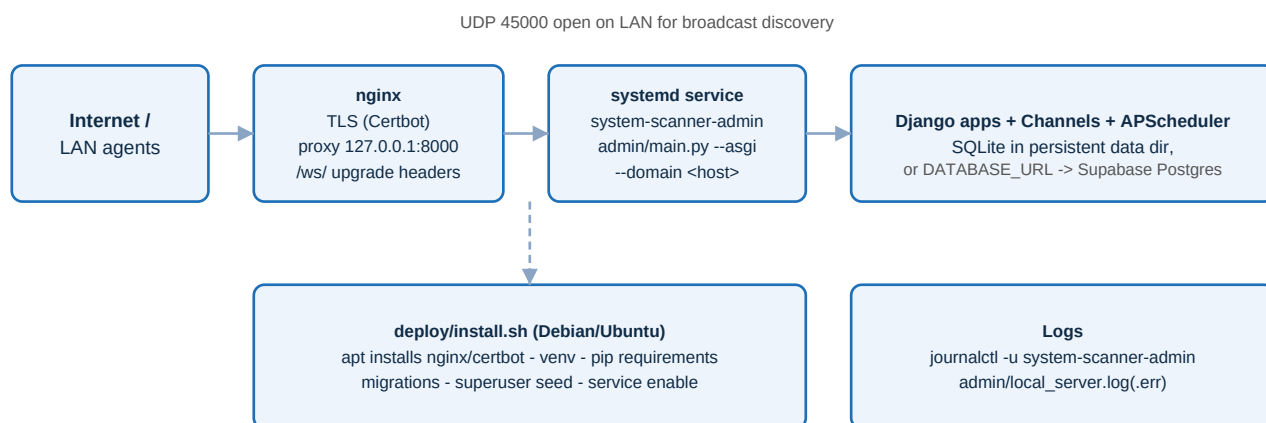


Figure 15 - VPS deployment topology

18. Deployment Architecture - Vercel Serverless (alternative)

Note: WebSocket consumers need ASGI (Daphne). The serverless WSGI path serves REST + pages; real-time push is designed for VPS mode. Agents still work via REST polling of endpoints.

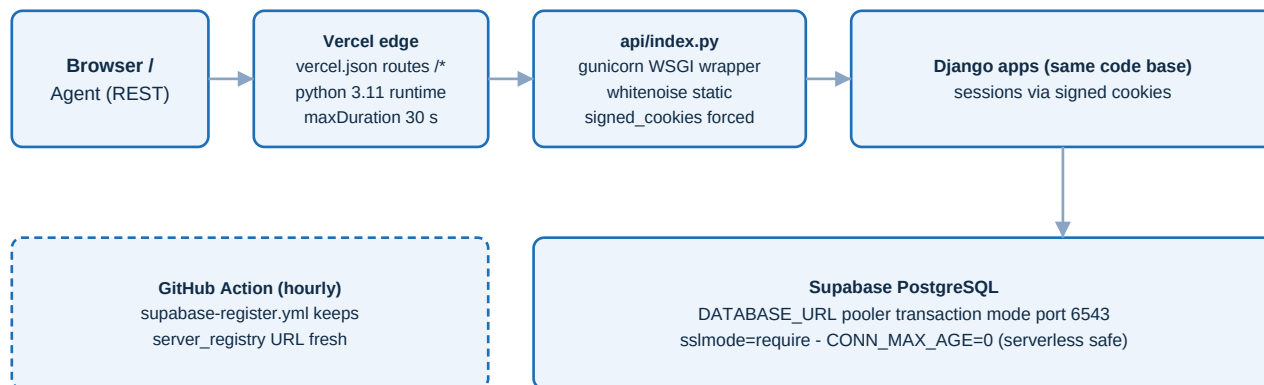


Figure 16 - Vercel serverless topology

19. Component Interaction Map

Source	Target	Mechanism	Payload / purpose
Agent	scanner_api	HTTPS POST	register, ping, scan results (JSON)
Agent	monitoring	WSS + HTTPS	heartbeats, metrics, events; receives scan_now
Agent	Supabase	REST (service key)	read/write server_registry discovery row
Dashboard JS	monitoring	WSS /ws/dashboard/	live device status, alerts feed
Dashboard JS	scanner_api	fetch (session cookie)	all CRUD pages, exports
Scheduler	monitoring models	ORM	stale/offline flips, periodic checks
Analytics engines	scan/metric tables	ORM reads	diffs, health, anomaly, predictions
Alert engine	intelligence models	ORM writes	Alert/Notification creation + rules eval
Report engine	all apps	ORM reads	PDF via ReportLab, Excel via openpyxl, CSV
GitHub Action	Supabase	REST	hourly server_registry refresh

Table 2 - Who talks to whom, and how

20. Design Decisions & Trade-offs

Decision	Rationale	Trade-off accepted
Monolith over microservices	Small team; single deployable; simpler ops	Scale-out requires channel-layer swap later
Signed-cookie sessions on serverless	DB-backed sessions impractical across cold starts	Cookie carries user id; kept short-lived and signed
JSON scan payloads + normalised tables	Raw fidelity for audits AND fast querying	Storage duplication
Statistical anomaly detection	No training data needed; explainable	Sensitive to noisy environments until tuned
HMAC per-agent secrets	Strong machine auth without PKI	Secret rotation must be manual today
In-memory channel layer	Zero extra infrastructure	Single-node WebSocket fan-out only
PyInstaller agent packaging	No Python needed on clients	Larger binary; AV false-positive risk handled by allowlisting

Table 3 - Key trade-offs